

20250717 日报

今日工作内容

1. 学习 CI/CD 相关知识。

CI/CD 通过自动化的“提交→构建→测试→部署”流水线，让每一次代码变更都能快速、安全、可重复地交付到生产环境，核心是用工具链与最佳实践把“人治”变成“机治”，实现质量、效率与稳定性的三重提升。

2. 仔细学习了流水线编排、触发器配置等。

- MR 前的预编译事件触发设定：

插件：[帮助文档](#)

版本：[①](#)

Git事件触发

重选

1.latest

代码库：*

☒ 按代码库选择

☐ 按代码库别名输入

drewjili/helloworld

事件类型：

☐ Commit Push Hook

☐ Tag Push Hook

☒ Merge Request Hook

☐ Merge Request Accept Hook

☐ Code Review Hook

☐ Issue Hook

☐ Note Hook

监听以下目标分支：

默认为所有分支，多个分支间以英文逗号分隔

排除以下目标分支：

多个分支间以英文逗号分隔

监听以下源分支：

多个路径间以英文逗号分隔

排除以下源分支：

多个路径间以英文逗号分隔

路径过滤类型：*

☒ 按路径前缀匹配

☐ 按通配符匹配

监听以下路径：

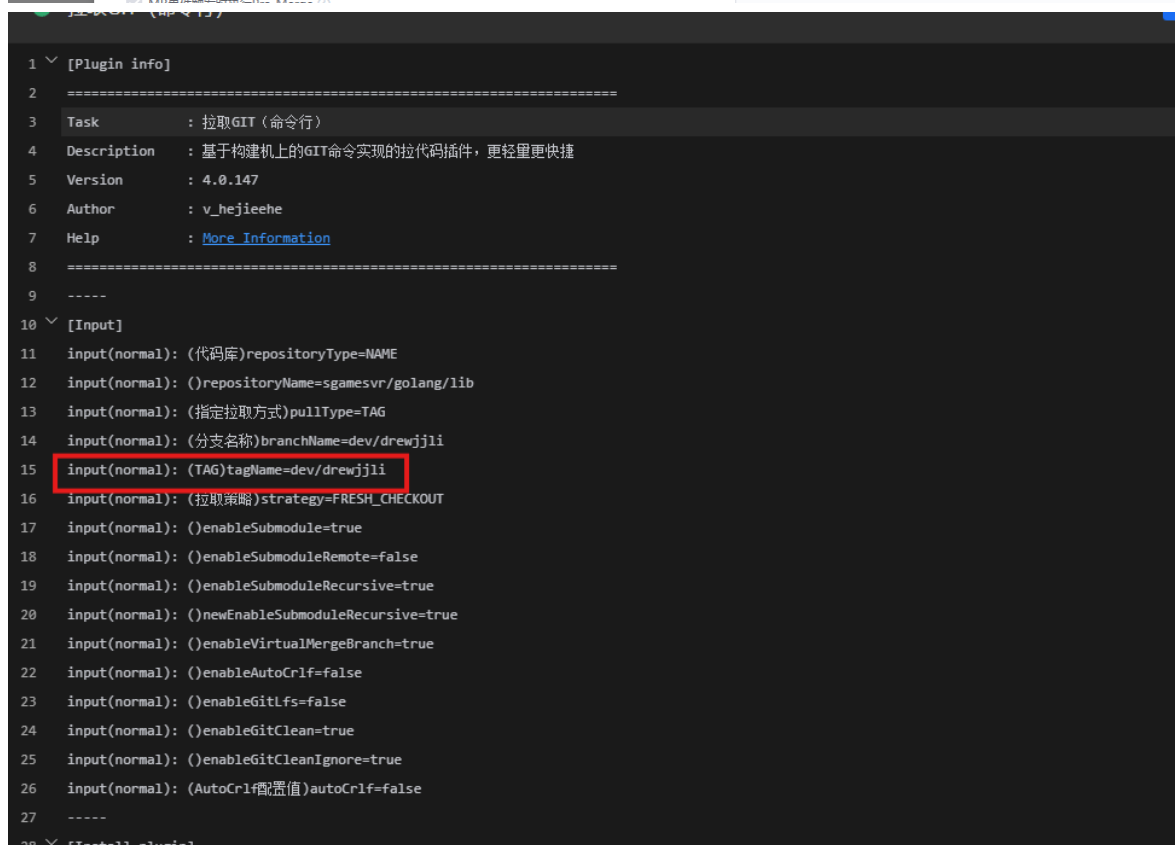
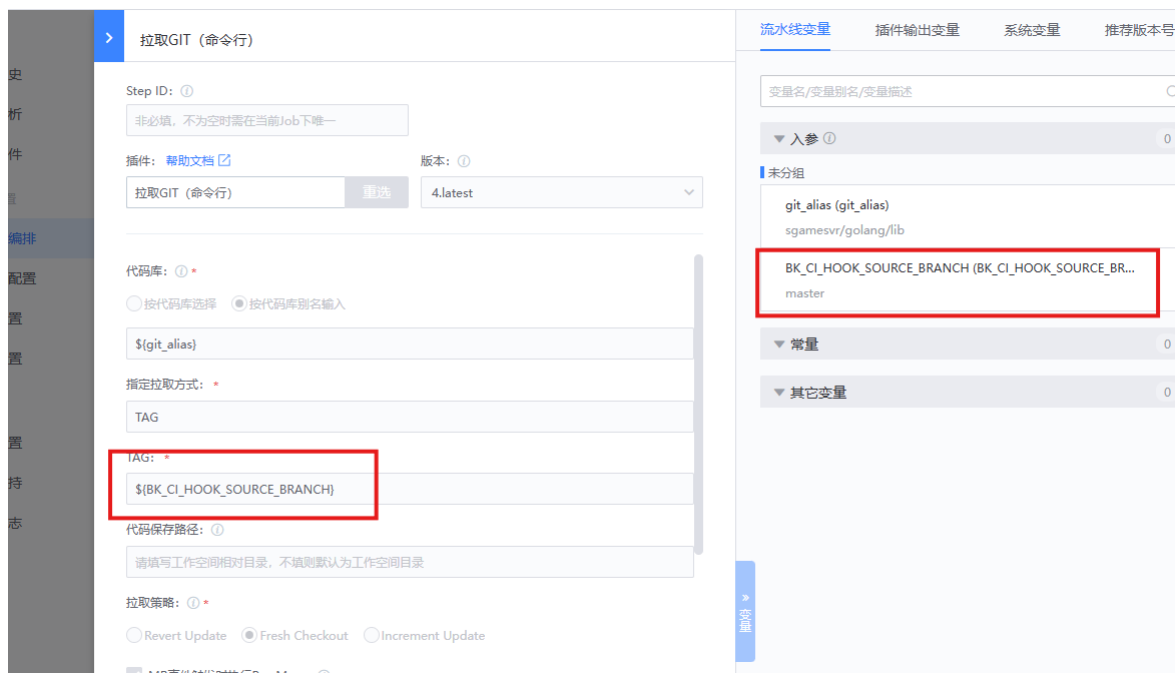
多个路径间以英文逗号分隔

排除以下路径：

多个路径间以英文逗号分隔

包含以下人员：[①](#)

- 疑惑：流水线编排 git 拉取时，设定了 TAG 表示的是 master，但是为什么在触发流水线时拉取到了开发分支，如下图：



原因：实际上流水线触发时 BK_CI_HOOK_SOURCE_BRANCH 表示的就是源分支，比如：当 MR 触发流水线时，那么 BK_CI_HOOK_SOURCE_BRANCH=MR 的源分支，而不是 master 分支。

3. 研究了在 mr 前的预编译脚本和 mr 成功后的后置脚本。

1、遇见问题：每次在流水线中构建后都会存在 3 个未被 Git 追踪的日志文件被 push 到

仓库，如下所示：

 `__init__.py`

 `b-018bae2797cd43fe88004876773ddafe-706b1f2f8158439abe5...`

 `b-018bae2797cd43fe88004876773ddafe-706b1f2f8158439abe5...`

 `b-018bae2797cd43fe88004876773ddafe-result.log`

 `b-094e57540fb2446e8a75d678b2ac6781-60ee74de4b894132b7...`

 `b-094e57540fb2446e8a75d678b2ac6781-60ee74de4b894132b7...`

 `b-094e57540fb2446e8a75d678b2ac6781-result.log`

 `b-0f924fd261124c5e8ee661fe5a09bded-6114efc3fe854463ad47...`

 `b-0f924fd261124c5e8ee661fe5a09bded-6114efc3fe854463ad47...`

 `b-0f924fd261124c5e8ee661fe5a09bded-result.log`

 `b-1b6240b0de514908a1c0d11639ea3204-e84e82ef6fbe4d29b1...`

 `b-1b6240b0de514908a1c0d11639ea3204-e84e82ef6fbe4d29b1...`

 `b-1b6240b0de514908a1c0d11639ea3204-result.log`

 `b-260df9ea0f954abb9d55c5850e03b68f-72035b183f3a4e68a4a...`

 `b-260df9ea0f954abb9d55c5850e03b68f-72035b183f3a4e68a4a...`

解决方法：在仓库的.gitignore 中添加*.log 即可。

2、当前的脚本逻辑如下：

- mr 后执行的前置预编译脚本：

```
cd $WORKSPACE
echo ${BK_CI_HOOK_SOURCE_BRANCH}

git fetch origin master
git fetch origin ${BK_CI_HOOK_SOURCE_BRANCH}
git checkout ${BK_CI_HOOK_SOURCE_BRANCH}

sh auto-make.sh
```

auto-make.sh :

```
git diff --name-only master | while read file; do dirname $file;
done | sort -u | grep -v "^. $" | while read dir; do echo $dir;
make dir=$dir; done
```

- mr 成功后执行的后置脚本，用于生成 pb 文件和打 tag，并推送到仓库：

```
cd $WORKSPACE
echo ${BK_CI_HOOK_SOURCE_BRANCH}

git fetch origin ${BK_CI_HOOK_SOURCE_BRANCH}
git checkout ${BK_CI_HOOK_SOURCE_BRANCH}

sh auto-make.sh

git add .
if ! git diff-index --quiet HEAD --; then
    git commit -m "post-build"
    git push origin ${BK_CI_HOOK_SOURCE_BRANCH}
else
    echo "no files changed, skip commit"
fi

sh increment-tag.sh
```

当前对于此需求的研究已经完成大部分的工作，我目前是在自己仓库构建了两个分支来测试 MR 预编译和后置编译的流水线，当前由于自己的流水线构建机上未安装 protoc 插件而无法编译 pb 程序，明天还需要继续调试。

明日待办：

1. 先在构建机上安装 protoc 插件，然后完成后续自测，若发现 bug 就继续调试脚本；